BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Trần Ngọc Ánh
MSSV / Lớp	3122410013_DCT1221

Mục tiêu học tập

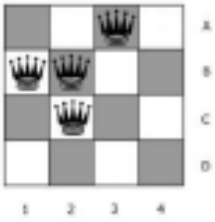
• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Bước 1: Xác định số cặp quân hậu vị xung đột

Q_1 = (A, 3); Q_2 = (B, 1); Q_3 = (B, 2); Q_4 = (C, 2)

Cặp Q_1, Q_3 và cặp Q2, Q4 cùng đường chéo.

Cặp Q_2, Q_3 cùng một hàng B

Cặp Q_3, Q_4 cùng một cột 2

Vậy số xung đột ban đầu là: h(S0) = 4

Bước 2: Chọn đường đi:

Thuật toán xét các trạng thái lân cận và chọn trạng thái có số xung đột nhỏ nhất. Một nước đi tốt nhất là di chuyển quân hậu Q_3 từ ô (B, 2) đến ô (D, 4). Có trạng thái mới là: $S_1 = \{(A, 3), (B, 1), (C, 2), (D, 4)\}$

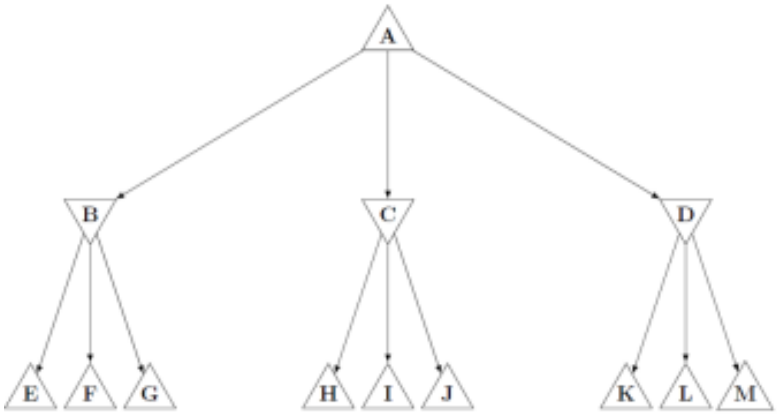
Trạng thái này, chỉ còn một cặp xung đột: $Q_2 = (B, 1)$ và $Q_4 = (C, 2)$, do nằm cùng đường chéo. Nên h(S1) = 1

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng
☐ B) Cực trị cục bộ (local optimum)
☐ C) Cực trị toàn cục (global optimum)
☐ D) Vai bằng (shoulder)
☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D
☐ C → D → B
☑ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Các nút không được thăm do bị cắt tỉa là: I, J, F, G

Các nút lá được thăm là K, L, M, H, E

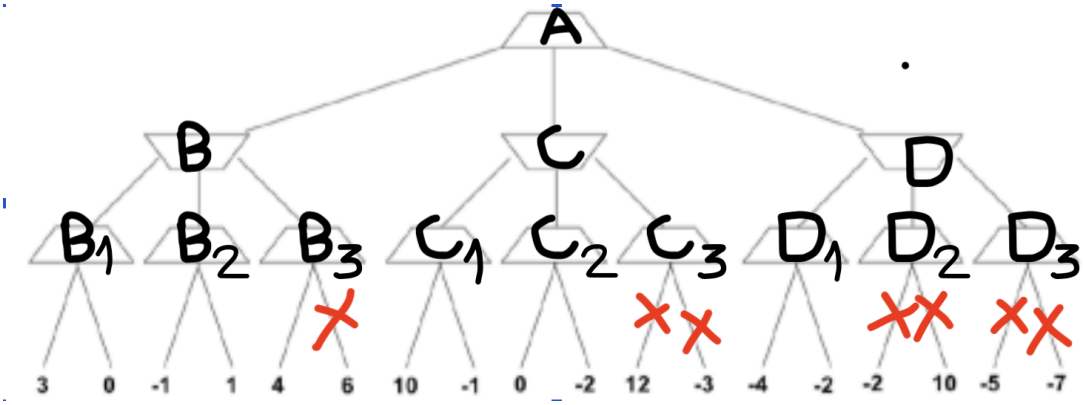
c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☑ Có

Alpha-Beta Pruning loại bỏ những nhánh không thể ảnh hưởng đến kết quả cuối cùng, nhưng không làm thay đổi giá trị Minimax của cây trò chơi. Iterative Deepening thực hiện tìm kiếm ở các độ sâu tăng dần. Khi kết hợp với Alpha-Beta Pruning, thuật toán có thể sử dụng kết quả của các lần tìm kiếm trước để cải thiện thứ tự duyệt, từ đó tăng khả năng cắt tỉa.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Bước 1: Duyệt cây B (nút MIN bên trái)

Tại B: $\alpha = -\infty, \beta = +\infty$

- $B_1 = max(3, 0) = 3$. Trả về B, có => $\beta_B = min(+\infty, 3) = 3$
- $B_2 = max(-1, 1) = 1$. Cập nhật tại B => $\beta_B = min(3, 1) = 1$
- $B_3 = max(4, 6) = 6$. Khi vào B_3, giới hạn là $\alpha = -\infty, \beta = 1$

  Thăm lá 4: nút MAX cập nhật $\alpha = \max(-\infty, 4) = 4$. Vì $\alpha \geq \beta$, tức $4 \geq 1$, ta cắt tỉa lá còn lại 6.

Do B MIN: $B = \min(3, 1, \geq 4) = 1$. Quay về gốc A: $\alpha_A = \max(-\infty, 1) = 1$. Kết quả nhánh B: giá trị 1.

Bước 2: Duyệt cây C (nút MIN ở giữa)

Tại C, kế thừa từ A: $\alpha = 1, \beta = +\infty$

- $C_1 = max(10, -1) = 10$. Cập nhật C => $\beta_C = min(+\infty, 10) = 10$
- $C_2 = max(0, -2) = 1$. Cập nhật tại C => $\beta_C = min(10, 0) = 0$

  Thăm 0: giá trị tốt nhất hiện tại của MAX là 0, nhưng giới hạn α kế thừa vẫn là 1.

  Thăm -2: giá trị nút I là $\max(0, -2) = 0$.

  Kiểm tra điều kiện cắt tỉa: $\beta_C \leq \alpha_A$ $với$ $0 \leq 1$ $(đúng)$

Vì vậy, toàn bộ cây con J, gồm hai lá $12, -3$, bị cắt tỉa.

Giá trị nút C: $C = \min(10, 0, \geq 0) = 0$. Quay về A: $\alpha_A = \max(1, 0) = 1$. Kết quả nhánh C: giá trị 0.

Bước 3: Duyệt cây D (nút MIN ở bên phải)

Đi vào D với: $\alpha = 1, \beta = +\infty$

- $C_1 = max(-4, -2) = -2$. Cập nhật D => $\beta_D = min(+\infty, -2) = -2$

  Kiểm tra điều kiện cắt tỉa: $\beta_D \leq \alpha_A$ với $-2 \leq 1 (đúng)$

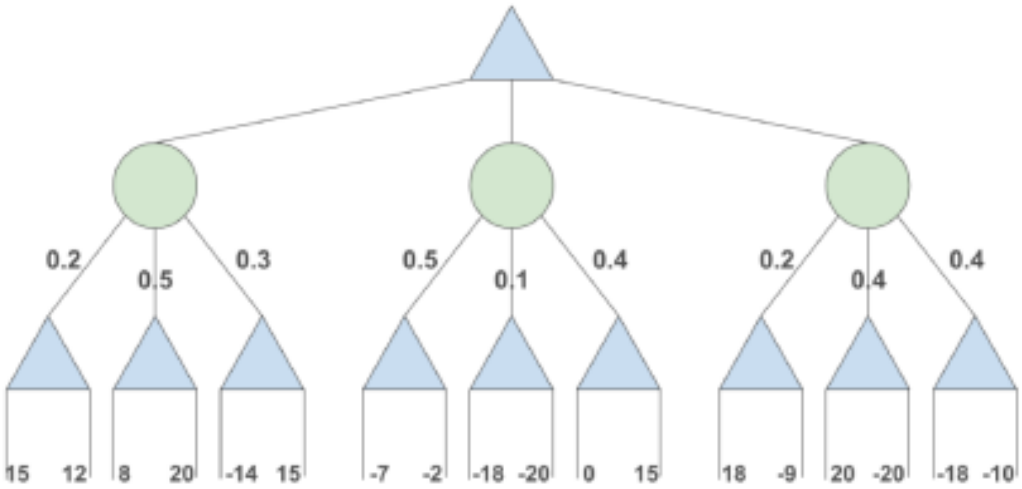
  Vì vậy, hai cây con D_2 *các lá* $-2.\ 10$ và D_3 *với các lá* $-5, -7$ bị cắt tỉa hoàn toàn

Giá trị trả về của D tại điểm cắt tỉa có thể được xem là một giới hạn trên -2, đủ để kết luận D không thể tốt hơn nhánh B có giá trị 1.

Nước đi tối ưu của MAX: Chọn nhánh B, nhánh trái. Các lá không được thăm do cắt tỉa: 6, 12, -3, -2, 10, -5, -7

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Tại nút MAX: $V_{MAX} = \max(V_1, V_2, ..., V_n)$

Tại chance node: $V_{Chance} = \sum_{i=1}^{n} p_i V_i$

Nút MAX	Giá trị 2 lá	Giá trị MAX
A_1	15, 12	15
A_2	8, 20	20
A_3	-14, 15	15
B_1	-7, -2	-2
B_2	-18, -20	-18
B_3	0, 15	15

C_1	18, -9	18
C_2	20, -20	20
C_3	-18, -10	-10

Chance node bên trái: Xác suất lần lượt là $0.2,\ 0.5,\ 0.3$, các giá trị nút MAX tương ứng là $15,\ 20,\ 15$.

$$V_L = 0.2(15) + 0.5(20) + 0.3(15) = 3 + 10 + 4.5 = 17.5$$

Chance node ở giữa: Xác suất lần lượt là $0.5,\ 0.1,\ 0.4$, các giá trị nút MAX tương ứng là $-2,\ -18,\ 15$.

$$V_M = 0.5(-2) + 0.1(-18) + 0.4(15) = -1 - 1.8 + 6 = 3.2$$

Chance node bên phải: Xác suất lần lượt là $0.2,\ 0.4,\ 0.4$, các giá trị nút MAX tương ứng là $18,\ 20,\ -10$.

$$V_R = 0.2(18) + 0.4(20) + 0.4(-10) = 3.6 + 8 - 4 = 7.6$$

Có $V_{root} = max(17.5,\ 3.2,\ 7.6) = 17.5$ => Chọn nhánh trái

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Tại nút MAX thông thường, nếu đã tìm thấy một giá trị tốt hơn giới hạn của nút cha, ta có thể cắt tỉa những nhánh không còn khả năng ảnh hưởng đến kết quả.

Tuy nhiên, tại chance node, giá trị được tính bằng tổng có trọng số của tất cả các nhánh: $V = \sum_i p_i V_i$

Muốn biết chính xác giá trị kỳ vọng, ta phải biết giá trị của tất cả các nhánh có xác suất khác 0.

Ví dụ, chance node bên trái có giá trị: $V_L = 0.2(15) + 0.5(20) + 0.3(15) = 3 + 10 + 4.5 = 17.5$

Nếu chưa biết giới hạn giá trị của các nhánh chưa xét, giá trị kỳ vọng cuối cùng có thể tăng hoặc giảm tùy theo kết quả của chúng. Vì thế, không thể bảo đảm cắt tỉa mà vẫn giữ nguyên kết quả chính xác.

Kết luận: Không thể cắt tỉa nhánh nào nếu không có thêm giới hạn về giá trị lá hoặc một điều kiện khác cho phép xác định chắc chắn rằng nhánh đó không thể làm thay đổi kết quả.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.

Có $-20 \le V_{lá} \le 20$

Duyệt chance node bên trái: $V_L = 17.5 \Rightarrow V_{root \ge 17.5}$

Xét chance node ở giữa: $V_{B_1} = max(-7,\ -2) = -2$

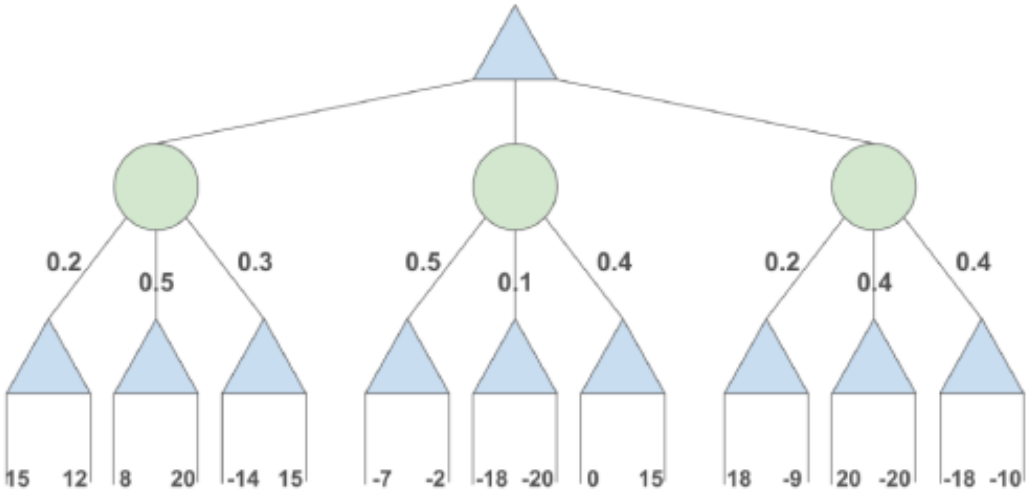
$V_M^{max} = 0.5(-2) + 0.1(20) + 0.4(20) = -1 + 2 + 8 = 9$ (<17.5, ngay cả trong trường hợp tốt nhất, chance node giữa vẫn không thể vượt qua nhánh trái của nút gốc). Cắt tỉa hai nút MAX con còn lại: (-18, -20) và (0, 15).

Xét chance node bên phải:

$V_{C_1} = max(18,\ -9) = 18 \Rightarrow V_R^{max} = 0.2(18) + 0.8(20) = 3.6 + 2 + 16 = 19.6$ (>17.5)

$V_{C_2} = max(20,\ -20) = 20 \Rightarrow V_R^{max} = 11.6 + 0.4(20) = 19.6$ (>17.5)

$V_3 = max(-18,\ -10) = -10 \Rightarrow V_R^{max} = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$ (<17.5)



Hình 5. Cây Expectimax dùng cho câu 4(c)

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	**Forward Checking (FC)**	**AC-3**
Phạm vi lan truyền ràng buộc	Chỉ kiểm tra các biến chưa gán có ràng buộc trực tiếp với biến vừa gán.	Kiểm tra các cung trong mạng ràng buộc và có thể lan truyền qua nhiều biến.
Thời điểm thực hiện	Thường thực hiện ngay sau mỗi lần gán giá trị cho một biến trong quá trình tìm kiếm.	Có thể chạy trước khi tìm kiếm để thu hẹp miền, hoặc chạy lại sau khi miền thay đổi.
Cách hoạt động	Loại bỏ các giá trị không tương thích trong miền của các biến chưa gán là láng giềng của biến vừa gán.	Kiểm tra từng cung $X_i \rightarrow X_j$; nếu giá trị của X_i không có giá trị hỗ trợ trong miền X_j, loại bỏ nó và kiểm tra lại các cung liên quan.
Khả năng phát hiện mâu thuẫn	Phát hiện khi miền của một biến chưa gán trở thành rỗng do các ràng buộc với biến vừa gán.	Có thể phát hiện mâu thuẫn sớm hơn nhờ lan truyền ràng buộc qua nhiều biến.
Chi phí tính toán	Thường thấp hơn, do phạm vi kiểm tra hạn chế.	Thường cao hơn, vì có thể phải kiểm tra lại nhiều cung.
Ưu điểm	Đơn giản, nhanh, phù hợp khi kết hợp với quay lui.	Lan truyền mạnh hơn, giúp giảm miền biến và hạn chế tìm kiếm vô ích.
Hạn chế	Có thể bỏ sót mâu thuẫn giữa các biến chưa gán với nhau.	Tốn thêm thời gian để duy trì tính nhất quán, nhất là khi mạng có nhiều cung.

Forward Checking có chi phí thấp nhưng lan truyền hạn chế; AC-3 kiểm tra tính nhất quán cung trên mạng ràng buộc, thường phát hiện mâu thuẫn sớm hơn nhưng tốn nhiều tính toán hơn.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV) : MRV giúp chọn biến có ít giá trị hợp lệ nhất, qua đó phát hiện thất bại sớm và giảm không gian tìm kiếm.

ii) Least Constraining Value (LCV) : LCV giúp chọn giá trị ít gây hạn chế nhất lên các biến khác, nhờ đó giảm khả năng dẫn đến ngõ cụt trong quá trình tìm kiếm.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: Mỗi máy bay là một biến: $X = \{A, B, C, D, E\}$ Trong đó A, B, C, D, E lần lượt đại diện cho 5 máy bay.

Miền giá trị: Mỗi máy bay được gán một khung giờ và một đường băng. Có thể biểu diễn giá trị của mỗi biến bằng cặp: $D(X_i) = \{1, 2, 3, 4\} \times \{QT, ND\}$

Ràng buộc đơn (Unary constraints)

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1 => $t_B = 1$
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi => $t_D \geq 3$
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2. $t_A \leq 2$

Ràng buộc nhị phân (Binary constraints)

- Ràng buộc thứ nhất: Máy bay D phải hạ cánh trước máy bay C cất cánh $t_D < t_C$

- Ràng buộc thứ hai: Không có hai máy bay nào được sử dụng cùng một khung giờ trên cùng một đường băng.

  Với mọi cặp máy bay khác nhau i, j: $(t_i, r_i) \neq (t_j, r_j) \Leftrightarrow t_i \neq t_j \vee r_i \neq r_j$

Bổ sung hai ràng buộc sau:

A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế: $r_A = r_B = r_C = QT$

D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa: $r_D = r_E = ND$

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Sau khi áp dụng các ràng buộc đơn ở câu a, ta có:

Biến	Miền ban đầu	Sau khi áp dụng ràng buộc đơn
A	{1, 2, 3, 4}	{1, 2}
B	{1, 2, 3, 4}	{1}
C	{1, 2, 3, 4}	{1, 2, 3, 4}

D	$\{1, 2, 3, 4\}$	$\{3, 4\}$
E	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4\}$

Các ràng buộc nhị phân:

1. $A \neq B$: hai máy bay quốc tế không được dùng cùng khung giờ trên đường băng quốc tế.
2. $A \neq C$: tương tự.
3. $B \neq C$: tương tự.
4. $D \neq E$: hai máy bay nội địa không được dùng cùng khung giờ trên đường băng nội địa.
5. $D < C$: máy bay D phải hạ cánh trước khi máy bay C cất cánh.

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

B1: Ràng buộc đơn

Có các miền ban đầu sau khi áp dụng ràng buộc đơn:

D(A) = {1, 2}; D(B) = {1}; D(C) = {1, 2, 3, 4}; D(D) = {3, 4}; D(E) = {1, 2, 3, 4}

B2: Xét $A \neq B$:

Vì $B = 1$, máy bay A không được ở khung giờ 1.

Loại bỏ giá trị 1 khỏi $D(A)$: $D(A) = \{2\}$

B3: Xét ràng buộc $B \neq C$ và $A \neq C$

Với $B = 1$ và $A = 2$, và do:

$C \neq B$, loại bỏ 1 khỏi miền của C.

$C \neq A$, loại bỏ tiếp 2 khỏi miền của C.

Vậy: $D(C) = \{3, 4\}$

B4: Xét ràng buộc $D < C$

Có : $D(D) = \{3, 4\}, D(C) = \{3, 4\}$

* Kiểm tra từng giá trị của D:

Nếu $D = 3$, có thể chọn $C = 4$, thỏa mãn $D < C$. Giữ lại 3.

Nếu $D = 4$, không tồn tại giá trị C nào lớn hơn 4. Loại bỏ 4.

Suy ra: $D(D) = \{3\}$

* Kiểm tra các giá trị của C:

Nếu $C = 3$, không tồn tại giá trị $D \in \{3\}$ sao cho $D < 3$. Loại bỏ 3.

Nếu $C = 4$, giá trị $D = 3$ thỏa mãn $D < C$. Giữ lại 4.

Suy ra: $D(C) = \{4\}$

B5: Xét ràng buộc $D \neq E$

Vì $D = 3$, máy bay E không được sử dụng khung giờ 3 => $D(E) = \{1, 2, 4\}$

AC-3 đã xác định được khung giờ duy nhất của bốn máy bay A, B, C, D. Máy bay E vẫn còn ba lựa chọn: $1, 2, 4$.

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

Theo MRV, B có miền chỉ chứa một giá trị nên được chọn đầu tiên. Áp dụng Forward Checking:

Do $A \neq B$, loại bỏ 1 khỏi miền A.

Do $B \neq C$, loại bỏ 1 khỏi miền C.

A chỉ còn một giá trị, nên được chọn tiếp, do $A \neq C$, loại bỏ 2 khỏi miền C.

Hai biến C, D cùng có 2 giá trị hợp lệ. Theo quy tắc phá hòa, chọn C trước vì C đứng trước D theo thứ tự chữ cái. Miền của C là $\{3, 4\}$, nên thử giá trị nhỏ nhất trước (C, 3). Ràng buộc yêu cầu: $D < C$. Nếu $C = 3$, ta phải có $D < 3$. Tuy nhiên, miền hiện tại của D là: $D(D) = \{3, 4\}$. Không có giá trị nào nhỏ hơn 3. Vì vậy: D(D) = $\varnothing$

Thử $C = 4$, (C, 4). Áp dụng Forward Checking cho ràng buộc $D < C$: $D < 4$

Trong miền $\{3, 4\}$, chỉ có $D = 3$ thỏa mãn. Do đó: $D(D) = \{3\}$

Miền của E vẫn là $\{1, 2, 3, 4\}$.

D chỉ còn một giá trị nên được chọn tiếp: (D, 3)

Do $D \neq E$, loại bỏ 3 khỏi miền của E: $D(E) = \{1, 2, 4\}$

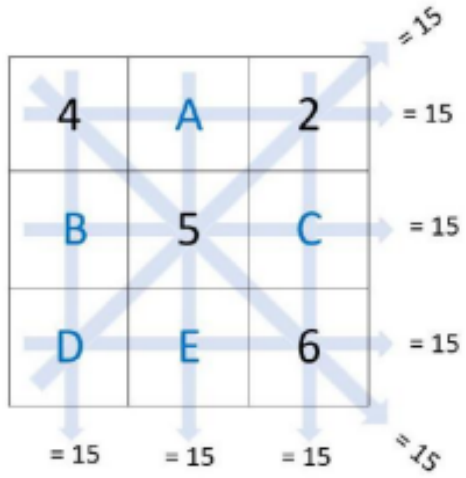
Chỉ còn biến E chưa được gán. Theo quy tắc chọn giá trị nhỏ nhất trước, ta chọn: (E, 1). Không có ràng buộc nào bị vi phạm. Tất cả các biến đã được gán giá trị hợp lệ, nên thuật toán kết thúc thành công.

Chuỗi phép gán: (B, 1) → (A, 2) → (C, 3) → (C, 4) → (D, 3) → (E, 1)

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n²}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Vì các ô 4, 2, 5, 6 đã biết, ta chỉ cần tạo biến cho năm ô còn lại: $X = \{A, B, C, D, E\}$

Miền ban đầu của mỗi biến chưa biết là: $D(A) = D(B) = D(C) = D(D) = D(E) = \{1, 3, 7, 8, 9\}$

Ràng buộc đơn chỉ liên quan đến một biến. Từ các hàng, cột và đường chéo có hai số đã biết, ta suy ra:

Hàng 1: 4 + A + 2 = 15 => A = 9

Cột 3: 2 + C + 6 =15 => C = 7

Đường chéo 2, 5, D: 2 + 5 + D = 15 => D = 8

Vậy các miền sau khi áp dụng ràng buộc đơn là:

D(A) = {9}, D(B) = {1, 3, 7, 8, 9}, D(C) = {7}, D(D) = {8}, D(A) = {1, 3, 7, 8, 9}

Ràng buộc nhị phân (tổng hàng, cột):

Từ hàng 2: $B + 5 + C = 15 \Rightarrow B + C = 10$

Từ cột 1: $4 + B + D = 15 \Rightarrow B + D = 11$

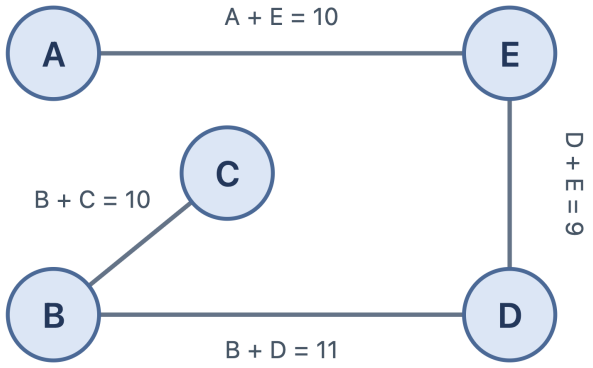
Từ hàng 3: $D + E + 6 = 15 \Rightarrow D + E = 9$

Từ cột 2: $A + 5 + E = 15 \Rightarrow A + E = 10$

Ràng buộc các số khác nhau: Mỗi số từ 1 đến 9 chỉ được sử dụng một lần. Vì vậy, với mọi cặp biến khác nhau X_i, X_j: $X_i \neq X_j$

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Các cạnh đồ thị: {AE, BC, BD, DE}



c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

* Xét ràng buộc B + C = 10. Vì C = 7 => B = 3 (loại bỏ các giá trị khác 3 ra khỏi miền B được D(B) ={3})

* Xét ràng buộc B + D = 11. Vì D = 8 => B = 3 (không phù hợp với ràng buộc trên, không cần loại bỏ thêm)

* Xét ràng buộc D + E = 9. Vì D = 8 => E = 1 (loại bỏ các giá trị khác 1 ra khỏi miền E được D(E) ={1})

* Xét ràng buộc A + E = 10. Vì A = 9 và E = 1 (ràng buộc được thoả mãn)

Có ma phương hoàn chỉnh:

Ma phương hoàn chỉnh

4	9	2
3	5	7
8	1	6

Nghiệm: A = 9, B = 3, C = 7, D = 8, E = 1

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

B1: Chọn biến theo MRV: Ellyse có miền nhỏ nhất, nên chọn Ellyse trước: $X_1 = Ellyse$

B2: Chọn giá trị theo LCV: chọn Hồng vì ít gây hạn chế hơn Đỏ

TH1: Ellyse = Đỏ => loại Đỏ khỏi các miền của biến Sunny và AJ

TH2: Ellyse = Hồng => loại Hồng khỏi miền của biến Sunny

Cập nhật miền: D(AJ) = {Đỏ, Vàng, Xanh lá}; D(Sunny) = {Đỏ, Vàng}; D(Ellyse) = {Hồng}

B3: Chọn biến tiếp theo theo MRV: Sunny có 2 giá trị, chọn Sunny: $X_2 = Sunny$

B4: Chọn màu cho Sunny theo LCV:

TH1: Sunny = Đỏ => loại Đỏ khỏi miền của AJ

TH2: Sunny = Vàng => loại Vàng khỏi miền của AJ

Hoà nhau. Chọn theo thứ tự đúng trước => chọn Đỏ. Cập nhật miền AJ: D(AJ) = {Vàng, Xanh lá}

B5: Gán màu cho AJ, có 2 lựa chọn là Vàng và Xanh lá. Không còn biến chưa gán nào bị ảnh hưởng bởi lựa chọn của AJ, nên hai giá trị có mức độ hạn chế bằng nhau => chọn Vàng.

Thứ tự các phép gán: (Ellyse, Hồng) → (Sunny, Đỏ) → (AJ, Vàng)

Số lần backtrack: 0

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

B1: Chọn biến đầu tiên: $X_1 = AJ$

B2: Chọn giá trị theo Most Constraining Value

AJ có ba màu: Đỏ, Vàng và Xanh lá. Ta xét số lựa chọn bị loại khỏi miền của các biến chưa gán:

Giá trị của AJ	Ảnh hưởng đến Sunny và Ellyse	Số giá trị bị loại
Đỏ	Loại Đỏ khỏi Sunny và Ellyse	2
Vàng	Loại Vàng khỏi Sunny	1
Xanh lá	Không loại giá trị nào	0

Most Constraining Value chọn Đỏ vì nó gây hạn chế nhiều nhất => AJ = Đỏ

Cập nhật miền: D(Sunny) = {Vàng, Hồng}; D(Ellyse) = {Hồng}

B3: Chọn Sunny làm biến tiếp theo vì có 2 giá trị: $X_2 = Sunny$

B4: Chọn giá trị cho Sunny theo Most Constraining Value: Sunny có hai lựa chọn là Vàng và Hồng

TH 1: Sunny = Vàng => Ellyse vẫn có thể chọn Hồng. Không có mâu thuẫn.

TH 2: Sunny = Hồng => Ellyse cũng chỉ còn màu Hồng, nhưng hai người không được chọn cùng màu. Vì thế, miền hợp lệ của Ellyse trở thành rỗng: D(Ellyse) = ∅

Giá trị Hồng gây hạn chế nhiều nhất vì nó loại bỏ lựa chọn duy nhất của Ellyse. Do đó, Most Constraining Value chọn: Sunny = Hồng => Nhưng đây là một phép gán thất bại, dẫn đến dead end.

B5: Backtrack và thử lại, quay llui khỏi phép gán: $Sunny = Hồng$ => thử $Sunny = Vàng$ => Ellyse = Hồng
Nghiệm cuối cùng: (AJ, Đỏ); (Sunny, Vàng); (Ellyse, Hồng). Số lần backtrack: 1
So sánh câu a và câu b:

Tiêu chí	Câu a: MRV + LCV	Câu b: Maximum Remaining Values + Most Constraining Value
Chọn biến đầu tiên	Ellyse	`AJ (giả sử ưu tiên thứ tự tên trong đề khi hòa)
Giá trị đầu tiên được chọn	Ellyse = Hồng	AJ = Đỏ
Có gặp dead end?	Không	Có
Số lần backtrack	0	1
Nghiệm cuối	AJ = Vàng, Sunny = Đỏ, Ellyse = Hồng	AJ = Đỏ, Sunny = Vàng, Ellyse = Hồng

Ở câu b, nếu chọn Sunny trước AJ khi hòa ở bước đầu, thuật toán có thể tìm được nghiệm mà không cần backtrack. Vì đề chỉ nêu thứ tự màu để phá hòa mà chưa quy định cách phá hòa khi chọn biến, số lần backtrack ở câu b phụ thuộc vào quy tắc chọn biến đó. Với quy ước chọn AJ trước, kết quả là 1 lần backtrack.

--- Hết ---